

COMP3000 Development Log

Week 1 - Team Based Learning

Logbook entry date: 28 July 2026

Before the workshop

I reviewed the Team Based Learning (TBL) overview and learned how the weekly preparation, readiness tests, and team activities would work.

During the workshop

I completed iRAT 1 and participated in the team activities. We also worked on our team contract, where we discussed expectations and how we would work together throughout the semester.

After the workshop

There was no major Assignment 1 development this week. My focus was on understanding the unit structure and the team-based workshop process.

What I learned

I understood how the TBL process would work and what was expected from me before and during future workshops.

Week 2 - Compiling and Interpreting Material

Logbook entry date: 4 August 2026

Before the workshop

I attended the Week 2 lecture and read Chapters 1 and 2 of *Crafting Interpreters*. I reviewed little languages, the compiler/interpreter pipeline, and the difference between compilers and interpreters.

During the workshop

I completed iRAT 2 and worked with my team using Regexp to experiment with regular expressions. We tried exact matches, alternatives, repeated patterns, and capture groups on sample text. We also explored extracting particular words from dialogue and discussed whether regular expressions are a complete language. We identified features such as capture groups for storing matched values and repetition using `*` and `+`, but found that ordinary regex does not provide the same general control flow as a general-purpose programming language.

After the workshop

I reviewed how the compiler/interpreter pipeline connects to the assignment. In particular, I understood that the scanner converts source code into tokens and the parser later turns those tokens into a structured tree.

What I learned

I learned that a language can be small and specialised for one domain. This later helped me think about designing a domain-specific language for river systems.

Week 3 - Lox

Logbook entry date: 11 August 2026

Before the workshop

I attended the Week 3 lecture, read Chapter 3 of *Crafting Interpreters*, and experimented with Lox before the workshop. I prepared for the iRAT and tRAT by reviewing Lox features such as expressions, variables, functions, and classes.

During the workshop

I completed iRAT 3 and participated in the tRAT with my team. I experimented with Lox in the online playground by writing and running small programs using variables, functions, loops, and output. For the application exercise, I used Lox to generate Logo turtle-graphics commands. I created my own geometric star design, compared it with my teammates' work and other workshop designs, and presented my result to the class.

[INSERT LOX PLAYGROUND SCREENSHOT HERE]

Figure 1. Week 3 Lox Playground practice.

[INSERT LOGO TURTLE GRAPHICS SCREENSHOT HERE]

Figure 2. Week 3 Logo turtle graphics.

After the workshop

I continued experimenting with Lox and reviewed how a program can generate another program. This helped me become more familiar with Lox syntax before starting the scanner and parser implementation in later weeks.

What I learned

I became more comfortable writing Lox programs and understood how loops and functions can generate repeated output instead of manually writing every command.

Week 4 - Scanning

Logbook entry date: 18 August 2026

Before the workshop

I attended the Week 4 lecture and read Chapter 4 of *Crafting Interpreters*. I studied how a scanner converts source characters into tokens, including lexical errors and lookahead, and prepared for the iRAT and tRAT.

During the workshop

I completed iRAT 4 and participated in the tRAT. My team then started designing the language for Assignment 1, focusing on how water flow over several days could be represented as a literal.

Our initial idea was to keep the literal simple by using a sequence of daily flow values, for example:

[10, 8, 6, 4, 2]

During the reporting period, I compared our idea with other teams. One design I remember conceptually used explicit day/value pairs, similar to:

flow{1:10, 2:8, 3:6, 4:4, 5:2}

I thought this was more elegant because the day associated with each flow value was immediately visible instead of being determined only by its position in a list. This made me think more carefully about making the meaning of a domain-specific literal visible in its syntax.

After the workshop

I continued working through the Chapter 4 Lox scanner and practised how source code is converted into tokens. I also started thinking about which keywords and symbols a river-specific language would require.

What I learned

I learned how scanning turns characters into tokens and how language-design decisions affect the scanner. The workshop also showed me that a literal can be designed to communicate both its value and its domain-specific meaning clearly.

Week 5 - Representing Code

Logbook entry date: 25 August 2026

Before the workshop

I attended the Week 5 lecture and read Chapter 5 of *Crafting Interpreters*. I followed the AST implementation in the book and learned how expressions can be represented as tree structures rather than a flat sequence of tokens. I also prepared for the iRAT and tRAT.

During the workshop

I completed iRAT 5 and participated in the tRAT. For the application exercise, my team explored how separate rivers could be combined to represent a larger river network.

We represented river combinations as expression trees and discussed how parentheses change the structure. For example:

A + (B + C)

means B and C combine first before joining A, while:

(A + B) + C

represents A and B combining first.

We also discussed different symbols for representing river connections. This made me realise that using **+** is simple, but a dedicated operator could make the topology of the river network clearer.

After the workshop

I continued Chapter 5 by working with AST classes and the Visitor pattern. I also used AstPrinter to make the tree structure visible, which helped me see how nested source expressions become nested AST nodes.

This later influenced my Assignment 1 design, where I used **<>** as a dedicated confluence operator rather than treating river combination as normal arithmetic addition.

What I learned

The main thing I learned was that the structure of the AST matters. Two expressions can contain the same rivers but describe different networks depending on how they are grouped.

I also understood why choosing syntax that reflects the underlying tree can make a domain-specific language easier to understand.

Week 6 - Parsing

Logbook entry date: 1 September 2026

Before the workshop

I attended the Week 6 lecture and read Chapter 6 of *Crafting Interpreters*. I reviewed recursive descent parsing, ambiguous grammars, precedence and associativity, and prepared for the iRAT and tRAT.

During the workshop

I completed iRAT 6 and participated in the tRAT. We worked on turning our river-language ideas from the previous week into grammar rules that could later be implemented in Java.

One water-flow example I worked through was:

`flow[10, 7, 3]`

I represented its structure with a grammar rule similar to:

`flowLiteral → "flow" "[" NUMBER ("," NUMBER) "]" ;*`

I then worked through how a parser could recognise `flow`, consume the opening bracket, read the numbers separated by commas, and finally consume the closing bracket.

We also discussed checking the grammar for ambiguity before implementing it.

After the workshop

I continued working with the Chapter 6 Lox parser and practised how grammar rules translate into recursive descent methods in Java.

This became the foundation for my Assignment 1 parser. I later extended the same approach to support my own `flow[...]` syntax, river declarations, confluences, grouped confluences, and the final `outlet` statement.

What I learned

I understood how the main stages of the language now connect:

Source → Tokens → Parser → AST

The most important thing I learned was that the grammar directly guides the structure of the parser. Each grammar rule can be translated into parser logic that consumes the expected tokens and builds the corresponding AST nodes.

Week 7 - Evaluating Expressions

Logbook entry date: 8 September 2026

Before the workshop

I attended the Week 7 lecture and read Chapter 7 of *Crafting Interpreters*. I reviewed the difference between expressions and values and learned how the Visitor pattern can be used to evaluate an AST by walking through its nodes.

During the workshop

I completed iRAT 7 and participated in the tRAT. My team discussed how the river expressions we had designed in previous weeks could actually be evaluated.

For example, we considered a simple combination such as:

RiverA + RiverB

and traced its tree:

- Evaluate RiverA
- Evaluate RiverB
- Combine the two resulting flow values
- Return the combined downstream flow

We also discussed where rainfall data should come from. For our workshop design, we considered keeping a small rainfall history directly in the evaluator because it made testing the tree-walking process easier.

After the workshop

I continued working through the Chapter 7 interpreter and practised tracing evaluation through AST nodes. I also reviewed runtime errors and why evaluation uses recursive Visitor calls.

This helped me understand the next stage of my RiverFlow language: the parser creates the river-network structure, while an evaluator could later walk that structure to calculate actual downstream flows.

What I learned

I learned the difference between an expression and its resulting value. For example, `1 + 2` is an expression, while `3` is the value obtained after evaluating it.

I also understood the overall pipeline more clearly:

Source → Tokens → AST → Evaluation → Value

Week 8 – Statements Part One

Logbook entry date: 15 September 2026

Before the workshop

I attended the Week 8 lecture and read Sections 8.1 and 8.2 of Chapter 8 of *Crafting Interpreters*. I reviewed how Lox moves from an expression-only language to a language containing statements. I also prepared my Assignment 1 progress for the optional Week 8 presentation.

During the workshop

I completed iRAT 8 and participated in the tRAT. We worked with statements, including expression statements, print statements and variable declarations.

One example I worked through was:

`var x = 3;`

print x;

This helped me distinguish between declaring a variable and later using that variable inside an expression.

I also discussed how adding a new statement affects several parts of an interpreter, including the grammar, parser, generated AST and Visitor methods.

Assignment 1 Presentation

For the optional Assignment 1 presentation, I submitted **RiverFlow Week 8.pdf** and presented my current RiverFlow language design.

I explained my current thinking about representing river responses, combining upstream rivers, and how the source code would be converted into an AST.

This was useful because I could compare my design with approaches being developed by other students and identify areas I wanted to improve before Submission 1.

[INSERT WEEK 8 ASSIGNMENT 1 PRESENTATION SUBMISSION SCREENSHOT HERE]

Figure 3. Week 8 Assignment 1 presentation submission.

After the workshop

I continued developing Assignment 1 using the Lox codebase. An important idea from this week was that RiverFlow should not only contain expressions.

A river declaration describes a river and its response, while an outlet identifies the final river of the network. This influenced my later use of statement nodes such as `Stmt.River` and `Stmt.Outlet`, while flow and confluence structures are represented as expressions.

What I learned

I learned that expressions produce values, while statements describe actions or larger pieces of program structure.

This distinction became important in my RiverFlow design:

- `flow[...]` and river confluences are expressions.
- `river ... { ... }` and `outlet ...;` are statements.

This gave my language a clearer structure than representing the entire river system as one large expression.

Learning Log - To Be Continued

This learning log currently records my progress through Week 8.

I will continue adding weekly entries as I complete the remaining COMP3000 workshops, readings, exercises, and Assignment 2 development.